Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Bu kayıt NetBeans ile canlı kodlanan uygulama oturumudur.
  - İlk 9 dakika IDE ve Discord sohbetidir. Asıl konu yaklaşık 00:09'da başlar.
- IDE'ler arası proje taşıma
  - IntelliJ, NetBeans ve Eclipse projelerinin yapılandırma dosyaları (Ant, Maven, Gradle) farklıdır. Proje doğrudan açılmayabilir.
  - En basit yol: `src` altındaki `.java` dosyalarını yeni projeye kopyalamak.
- Örnek: Kitap, Yazar ve Kütüphane sınıfları
  - Önce alanlara test sınıfından doğrudan erişilerek başlandı. Sonra bu yaklaşımın nesne yönelimli mantığa aykırı olduğu gösterildi.
  - Alanlar `private` yapıldı. Constructor, get metotları ve tanıtım metotları yazıldı.
  - Kitap constructor'ı aşırı yüklendi (overloading). `this(...)` ile kod tekrarı önlendi.
- Referans tipleri
  - `new` ile üretilen her nesne referans tiplidir. Nesneyi doğrudan yazdırınca adres benzeri bir değer görünür.
  - == iki nesnenin içeriğini değil, referansını karşılaştırır.
  - Alanların varsayılan değerleri: sayısal alanlar `0`, referans alanlar `null`.
- String birleştirme ve tip dönüşümü
  - `"AAA" + 3` sonucu `"AAA3"` olur. `String` yanındaki değeri "yutar".
  - `null` bir String'e eklenince `"null"` metni olur.
  - Küçük tip büyük tipe atanabilir (`byte` -> `short` -> `int`). Tersi derleme hatası verir.
- `static` metot ve alan
  - Statik metot nesne üretmeden sınıf adıyla çağrılır (`Kitap.f()`, `Math.abs()`, `Math.PI`).
  - Statik metodun içinden statik olmayan alana erişilemez.
  - Statik alan bütün nesnelerde ortaktır. Her çağrıda kaldığı yerden artar.
- `private` set metodu, `finalize()` ve çöp toplayıcı (garbage collector)
- Kütüphane sınıfı: kitap dizisi, kapasite, `kitapEkle`, `kutuphaneListele`, toplu ekleme (`kitapEkle(Kitap[])`)
- `Scanner` ile klavyeden kapasite okuma, nesne dizisi oluşturma ve dizi `length` özelliği

Hocanın Özellikle Vurguladığı Kısımlar

- Nesne yönelimli programlamada amaç soyutlamadır.
  - Kullanıcı (test sınıfı) kitabın ID'sine veya yazarın adına doğrudan erişip değiştirememeli.
  - Kütüphaneyi kullanan kişi kitabın nasıl eklendiğini bilmek zorunda değildir. Arkada dizi mi, LinkedList mi kullanıldığı onu ilgilendirmez.
- Alanlar `private` olmalı.
  - Bir kez atanıp sonra değişmemesi gereken alanlar (arabanın şase numarası gibi) yalnızca constructor ile atanır.
  - Bu alanların public set metodu olmamalı. Açık set metodu "arka kapı" gibidir.
- Bir sınıfta zaten yazılmış bir metodu tekrar yazmayın, çağırın.
  - Kitap tanıtılırken yazarın bilgisi `this.yazar.yazarTanit()` ile alındı.
  - İkinci constructor ilkini `this(id, ad)` ile çağırdı.
- Senaryoyu kurgularken hangi nesnenin önce var olması gerektiğini düşünün.
  - Kitap yazmak için önce yazar gerekir. Kodda da önce `Yazar` sınıfı yazıldı.

- Constructor overloading, Java'da varsayılan parametre olmadığı için kullanılır.
  - C# ve Python'da varsayılan parametre vardır. Java'da yoktur.
  - Kullanıcı `null` göndermeyi unutabilir. Overloading bu riski kaldırır.
- `String` ile `null` toplamı mülakatlarda sorulur.
- Alıştırma önerisi: Bir kitabın birden fazla yazarı olsun.
  - `yazar` alanı bir diziye dönüşür. `kitapTanit` içinde yazarlar döngüyle yazdırılır.

Kısa Tekrar Notları

- Java'da `String` büyük harfle yazılır. Küçük `string` yoktur.
- Nesne yazdırınca varsayılan çıktı `paket.SinifAdi@hashkod` biçimindedir.
- == nesnelerde referans karşılaştırır. İçerikleri aynı olan iki ayrı nesne için sonuç `false` olur.
- Varsayılan constructor alanları sıfırlar: `int` için `0`, referans için `null`.
- Yerel değişken ilk değersiz kullanılırsa derleme hatası: "variable might not have been initialized".
- `this.alan` sınıfın alanını, çıplak ad parametreyi gösterir.
- `this(...)` aynı sınıfın başka constructor'ını çağırır. Hoca ilk satırda yazdı; sürüm farkı için ders sonrası notlara bakın.
- NetBeans: Refactor > Encapsulate Fields ile get/set metotları otomatik üretilir. Bir öğrencinin aktardığına göre IDE'nin Generate menüsünden constructor, `toString`, `equals` da üretilebilir.
- Statik metot içinden statik olmayan alana erişim hatası: "non-static variable cannot be referenced from a static context".
- `Math.abs`, `Math.PI`, `Math.E` statiktir. Nesne üretmeden kullanılır.
- `finalize()` nesne çöp toplayıcı tarafından silinirken çağrılır. `System.gc()` çöp toplayıcıyı çalıştırmayı ister.
- Dizi uzunluğu `dizi.length` ile alınır (parantez yok).
- Klavyeden okuma: `Scanner sc = new Scanner(System.in); int n = sc.nextInt();` (`import java.util.Scanner;`).
- `byte -> short -> int -> long` ve `float -> double` yönünde atama serbesttir. Ters yön derleme hatasıdır.

Detaylı Açıklamalar

1. Yazar ve Kitap sınıfları (son hâli)

Kod, ders sonundaki ekran karelerinden alındı. Çıktı biçimi karede görülen biçimle aynıdır.

```
public class Yazar {
    private int id;
    private String ad;

    public Yazar(int id, String ad) {
        this.id = id;
        this.ad = ad;
    }

    public int getId() { return this.id; }
    public String getAd() { return this.ad; }

    public void yazarTanit() {
        System.out.println("[YAZAR ID:" + this.getId() + "--->YAZAR AD:" + this.getAd() + "]")
    }
}

public class Kitap {
    private int id;
    private String ad;
    private Yazar yazar;   // tek yazar kabulü
```

```
    public Kitap(int id, String ad) {
        this.id = id;
        this.ad = ad;      // yazar varsayılan olarak null kalır
    }

    public Kitap(int id, String ad, Yazar yazar) {
        this(id, ad);      // kod tekrarı yok
        this.yazar = yazar;
    }

    public void kitapTanit() {
        System.out.println("[ID:" + this.id + "--->AD:" + this.ad + "]");
        if (this.yazar != null)
            this.yazar.yazarTanit();
        else
            System.out.println("--->Yazar bulunamadi");
    }
}
```

Test sınıfında:

```
Yazar y1 = new Yazar(1, "ALI");
y1.yazarTanit();
Kitap k  = new Kitap(1, "JAVA-1");       // yazarsız
Kitap k2 = new Kitap(2, "JAVA-2", y1);   // yazarlı
k.kitapTanit();    // [ID:1--->AD:JAVA-1]  --->Yazar bulunamadi
k2.kitapTanit();   // [ID:2--->AD:JAVA-2]  [YAZAR ID:1--->YAZAR AD:ALI]
```

- Kitap ve yazar arasında bir referans ilişkisi vardır. `k2` nesnesi `y1` nesnesinin adresini tutar.
- Hoca, dersin başında alanlara `y1.id = 1;` diye doğrudan değer atadı. Sonra bu kodu sildi. Gerekçe: test sınıfı (kullanıcı) bu alanları değiştirememeli.

2. Referanslar ve ==

- `Yazar y1 = new Yazar();` satırında önce `new` nesneyi bellekte üretir. Sonra adresi `y1`'e atanır.
- `System.out.println(y1);` nesnenin içeriğini değil, `paket.Yazar@1b6d3586` gibi bir değer yazar.
- İçerikleri aynı iki nesne (`y1` ve `y2`) `==` ile karşılaştırılınca `false` döner. Bellekte iki ayrı nesne vardır.

3. String birleştirme ve tip dönüşümleri

- `+` operatörünün bir tarafı `String` ise diğer taraf da `String`'e dönüştürülür.
  - `"AAA" + 3` -> `"AAA3"`, `"AAA" + 3.0` -> `"AAA3.0"`.
  - `x` değeri `"AAA3.0"` iken `x += "BBB" + null;` -> `"AAA3.0BBBnull"`. Java ile denendi.
- İşlem sırası soldan sağadır. `1 + 2 + "A" + 1 + 2` -> `"3A12"`. Önce sayılar toplanır, String görülünce birleştirme başlar.
- Referans tipli yerel değişkene ilk değer olarak `null` verilebilir: `Yazar ye = null;`.
- Sayısal tiplerde küçükten büyüğe atama otomatiktir:

```
byte b = 1;
short s = 3;
s = b;      // geçerli
int c = s;  // geçerli
// b = s;   // derleme hatası: incompatible types, possible lossy conversion
```

4. static

```
public static void f() { ... }     // Kitap içinde
Kitap.f();                         // tercih edilen çağrı
```

```
k.f();                              // nesne üzerinden de çalışır ama önerilmez
```

- Statik alan sınıfa aittir. Örnekte `static int ids = 0;` alanı her `f()` çağrısında artırıldı. Çıktı `0, 1, 2, 3` oldu.
- `System.out` da statik bir alandır. `System` sınıfından nesne üretmeden kullanırız.
- `Math` sınıfındaki `abs` metodu `int`, `long`, `float`, `double` için aşırı yüklenmiştir. `Math.PI` ve `Math.E` hem `static` hem `final`'dır, değiştirilemez.

5. `private set`, `finalize` ve çöp toplayıcı

- `private void setId(int id)` dışarıdan çağrılamaz. Yalnız sınıfın içinde kullanılır.
- Örnek: `k = k2;` atamasından sonra ilk kitap nesnesini gösteren referans kalmaz. Nesne "başıboş" kalır ve çöp toplayıcı onu siler.
- Silinmeden önce `finalize()` çağrılır:

```
@Override
public void finalize() {
    System.out.println(this.id + " nolu kitap silindi");
}
```

- `System.gc()` çağrılınca çıktıda "1 nolu kitap silindi" görüldü.
- `@Override` bir annotation'dır. Metodun üst sınıftaki (burada `Object`) bir metodu ezdiğini gösterir.

6. Kütüphane sınıfı

```
public class Kutuphane {
    int kapasite;
    Kitap[] kitaplar;
    int kitapSayisi;

    public Kutuphane(int kapasite) {
        kitapSayisi = 0;
        this.kapasite = kapasite;
        kitaplar = new Kitap[kapasite];   // yalnız referans dizisi açılır
    }

    public boolean kitapEkle(Kitap k) {
        if (this.kitapSayisi < this.kapasite) {
            kitaplar[this.kitapSayisi] = k;
            kitapSayisi++;
            return true;
        }
        return false;
    }

    public void kutuphaneListele() {
        for (int i = 0; i < kitapSayisi; i++)
            kitaplar[i].kitapTanit();
    }
}
```

- `new Kitap[4]` dört kitap nesnesi üretmez. Dört tane `null` referans içeren bir dizi üretir. Hoca bunu C'deki çift pointer'a benzetti.
- Listeleme kapasiteye kadar değil, `kitapSayisi`'na kadar yapılır. Yoksa boş (`null`) hücrelerde hata olur.
- Döngüyle nesne üretip ekleme:

```
Kitap[] kitaplar = new Kitap[4];
for (int i = 0; i < 4; i++) {
    kitaplar[i] = new Kitap(i, "Java" + i, y1);
```

```
    kutuphane2.kitapEkle(kitaplar[i]);
}
```

- Bir metodun döndürdüğü `boolean` değeri kullanmak zorunlu değildir. Java bunun için hata vermez.

Ders sonrası notlar

- **Varsayılan değerler:** Derste `0, 0, 0` çıktısı için "ad ve yazar referans tipli olduğu için değerleri 0" dendi. Doğrusu: `int` alan `0`, `String` ve `Yazar` alanları `null` olur. Çıktı `0 null null` biçimindedir (Java 25 ile denendi).
- **Bir dosyada birden fazla sınıf:** Derste "Java'da bunu yapamıyorsunuz" dendi. Bir `.java` dosyasında birden fazla sınıf olabilir. Kısıt şudur: dosyada en fazla bir `public` üst düzey sınıf olur ve adı dosya adıyla aynı olmalıdır.
- **`this(...)` çağrısı:** Derste doğru söz dizimi bir süre arandı. Doğrusu `this(id, ad);` biçimindedir. JDK 25'te `this(...)` öncesinde başka deyim yazılabilir: Java ile denendi, derlendi. `-source 21` ile aynı kod hata veriyor; dersin yapıldığı dönemdeki sürümlerde ilk satır olması zorunluydu.
- **`String` temel tip değildir:** Hoca "integer gibi string gibi primitif şeyler" dedi. `int` temel tiptir, `String` sınıftır.
- **Nesne yazdırma:** `println` nesnenin `toString()` metodunu kullanır. Varsayılan sonuç `sınıfAdı@hashCode` biçimindedir; bu doğrudan bellek adresi değildir.
- **Varsayılan constructor:** Hoca onu Java Virtual Machine'in çalıştırdığını söyledi. Varsayılan constructor'ı derleyici üretir.
- **`finalize` görünürlüğü:** Hoca `public` yazdı, `Object`'teki hali `protected`'tır. Görünürlüğü artırarak ezmek geçerlidir: Java ile denendi, derlendi.
- **Statik yerel değişken:** Hoca metodun içinde `static int id = 1;` yazınca "illegal start of expression" hatası aldı. Sebep: Java'da C'deki gibi statik yerel değişken yoktur. `static` alan yalnız sınıf düzeyinde tanımlanır. Hoca değişkeni sınıf düzeyine taşıyarak sorunu çözdü.
- **`finalize()`:** Java 9'dan beri kullanımdan kaldırılmış (deprecated) durumdadır. Çağrılacağı garanti değildir. `System.gc()` yalnız bir istektir, çöp toplayıcının hemen çalışmasını garanti etmez. Kaynak serbest bırakmak için `try-with-resources` ve `AutoCloseable` tercih edilir.
- **`length`:** Derste dizinin `length` değerinin "dinamik bağlama sayesinde" geldiği ve `Object` sınıfında tanımlı olabileceği söylendi. Doğrusu: `length`, her dizi nesnesinin `public final` bir alanıdır. Dizi oluşturulurken belirlenir. Dinamik bağlamayla (dynamic binding) ilgisi yoktur.
- **`append`:** Hoca listeye eklemeyi "append" diye anlattı. Java'da `ArrayList` için karşılığı `addAll`'dur.
- **Toplu ekleme hatası (01:18-01:26):** Hoca `kitapEkle(Kitap[] k)` metodunu `kapasite - kitapSayisi >= k.length` şartıyla yazdı. Kapasite 4 iken kütüphane zaten 4 kitapla doluydu. Şart sağlanmadı, metot `false` döndü ve hiçbir kitap eklenmedi. Bu yüzden ikinci listede de ID'ler `0, 1, 2, 3` göründü. Hoca bunu önce "`i+5` hesaplanmadı" diye yorumladı, ama sorun şarttaydı.
  - Hoca sonra şartı kaldırıp `this.kitaplar = k;` yazdı. Bu satır diziyi kopyalamaz. Eski kitapları atar ve yalnız yeni dizinin referansını tutar. `kitapSayisi` de güncellenmez.
  - Ekleme yerine kopyalama doğru yoldur. Kapasite yeterliyse yeni kitaplar mevcutların sonuna tek tek eklenir:

```
public boolean kitapEkle(Kitap[] k) {
    if (this.kapasite - this.kitapSayisi < k.length)
        return false;
    for (Kitap x : k)
        kitapEkle(x);
    return true;
}
```

- Kapasite 8 ile denendiğinde 8 kitap listelendi: ID'ler `0..3` ve `5..8`. Adlar `"Java" + i` olduğu için ikinci grubun adları da `Java0..Java3` olur.
- **Tip dönüşümü sorusu:** Bir öğrenci "s sıfır olsa da olmuyor değil mi?" diye sordu. De-

ğişkenler için cevap doğrudur. İstisna: derleme zamanında bilinen sabitlerde daraltma yapılabilir. Örnek: `final short s = 3; byte b = s;` derlenir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.